Guia de teste da avaliação: RAG against the machine

Comandos a correr, o que acontece em cada um e que funções do teu código são chamadas

Base deste guia. Li a evaluation sheet, o Makefile, o README, os 3 scripts de exame (exam_retrieval.sh, exam_answer.sh, exam_edge_cases.sh) e todo o código em src/student (__main__.py, indexer.py, chunker.py, generator.py, models.py). Os nomes de funções e a ordem vêm do código. **Não consigo ver nem correr** o binário moulinette; por isso o que ele calcula por dentro está marcado como tal. Os exemplos numéricos são ilustrativos.

1. Comandos pela ordem da evaluation sheet

#	Comando	Questão da sheet	Notas
1	make install	Q1 (uv sync)	Apaga .venv e uv.lock e recria tudo.
2	make prepare	Q1 (unzip datasets privados)	Extrai vLLM, datasets, exams, moulinette.
3	make index	Pré-requisito	Constrói o índice BM25.
4	make exam-retrieval	Q2 (background) e Q7a-Q7d	Re-indexa e avalia. Deixa correr.
5	uv run python -m src search "How to configure OpenAI server?" --k 10	Q5	Demo ao vivo da pesquisa.
6	uv run python -m src answer "How to configure OpenAI server?" --k 10	Q6	Demo da resposta (Qwen3-0.6B).
7	make exam-answer	Q8 e Q8b	Precisa dos resultados do passo 4.
8	make exam-edge-cases	Q11	4 testes, cerca de 1 minuto.
+	make lint e grep -rn "moulinette" src --include="*.py"	Q3 e Guidelines	O grep não devolveu nada no teu código.

- Na sheet, o passo 4 corre em segundo plano enquanto fazes Q3 a Q6. Como o exam_retrieval.sh **volta a indexar primeiro** (o README indica cerca de 9 s), espera que a fase de indexação acabe antes da demo ao vivo, para não leres o índice enquanto está a ser reescrito. (Precaução minha, não vem do código.)
- A sheet usa `python -m src`. O ficheiro src/__main__.py apenas chama `student.__main__.main`, por isso é exatamente a mesma CLI. Os targets do Makefile passam `--module-name student`.

2. Mapa de quem chama quem

```
python -m student <comando>
  __main__.main()  ->  fire.Fire(RAGSystem)  ->  RAGSystem.<comando>()

index           -> BM25Index.build() -> collect_files -> should_skip_file
                                     -> get_file_chunks -> chunk_python_file | chunk_text_file
                                          (_split_by_size, _hard_split, _enforce_max_size)
                                     -> tokenize -> BM25Okapi
                -> BM25Index.save()
search          -> _validate_k -> _load_index (BM25Index.load) -> BM25Index.search -> tokenize
search_dataset  -> _validate_k -> _load_index -> _load_json_model(RagDataset)
                -> BM25Index.search (por pergunta) -> MinimalSource / MinimalSearchResults
                -> StudentSearchResults -> _write_json
answer          -> _validate_k -> _load_index -> BM25Index.search
                -> AnswerGenerator.load -> AnswerGenerator.generate -> _build_prompt
answer_dataset  -> _load_index -> _load_json_model(StudentSearchResults)
                -> AnswerGenerator.generate (por pergunta) -> MinimalAnswer
                -> StudentSearchResultsAndAnswer -> _write_json
evaluate        -> _load_json_model (x2) -> _compute_recall_at_k -> _source_found -> _iou
```

3. Comando a comando

3.1 make install

São comandos de shell, sem funções Python: `rm -rf .venv uv.lock`, `uv venv`, `uv sync`. Cria o ambiente virtual e instala as dependências do pyproject.toml (fire, pydantic, rank-bm25, tqdm, transformers, torch, accelerate, flake8, mypy). Cobre o requisito da Q1 de o uv sync terminar sem erros.

3.2 make prepare

Também só shell. Por ordem:

- Cria as pastas data/raw, data/datasets/AnsweredQuestions, data/datasets/UnansweredQuestions e as duas de output.
- Extrai vllm-0.10.1.zip para data/raw (se ainda não existir). É o repositório que vai ser indexado.
- Extrai datasets_public.zip e copia os JSON para data/datasets/AnsweredQuestions e UnansweredQuestions.
- Chama o target interno _stage-eval-materials: extrai moulinette_pkg.zip, exams.zip e datasets_private.zip, dá permissão de execução aos scripts e copia os datasets privados para onde o exam_retrieval.sh os procura.
- No macOS cria um wrapper Docker para o moulinette-ubuntu, que é um binário Linux.

3.3 make index

Corre `uv run python -m student index --max_chunk_size 2000`. Já dominas este passo, mas aqui está a cadeia real de funções para a poderes referir:

#	Função (ficheiro)	O que faz
1	`main()` (__main__.py)	Entrega a classe RAGSystem ao Fire, que transforma o argumento **index** numa chamada ao método.
2	`RAGSystem.index()`	Resolve o caminho data/raw/vllm-0.10.1, cria o BM25Index com o max_chunk_size, chama build() e depois save().
3	`BM25Index.build()` (indexer.py)	Orquestra a indexação: lista ficheiros, lê cada um, divide em chunks, tokeniza e cria o BM25.

#	Função (ficheiro)	O que faz
4	`collect_files()` e `should_skip_file()`	os.walk ao repositório. Ignora pastas como .git, build, csrc, benchmarks e ficheiros que não sejam .py, .md, .rst, .txt, .yaml, .yml, .toml, .cfg, .ini, .sh.
5	`get_file_chunks()` (chunker.py)	Escolhe a estratégia pela extensão: .py usa a de código, o resto usa a de texto.
6	`chunk_python_file()`	**Estratégia 1 (código)**: ast.parse, corta nas funções e classes de topo. Se o ficheiro tiver erro de sintaxe, usa a de texto.
7	`chunk_text_file()`	**Estratégia 2 (texto/markdown)**: separa por linhas em branco e junta parágrafos adjacentes até ao limite.
8	`_split_by_size()`, `_hard_split()`, `_enforce_max_size()`	Apoio: partem regiões grandes por linhas, depois por caracteres, e garantem que nenhum chunk passa de 2000.
9	`tokenize()` (indexer.py)	Minúsculas, separa por pontuação, parte snake_case em sub-palavras e mantém o token completo.
10	`BM25Okapi(corpus)` (rank_bm25)	Constrói o modelo de pontuação com a lista de tokens de cada chunk.
11	`BM25Index.save()`	Grava bm25.pkl, chunks_meta.json, chunk_texts.pkl em data/processed/bm25_index e o manifest.json em data/processed/chunks.

> ***O que dizer:*** *"Tenho duas estratégias de chunking: uma por AST para Python, que respeita funções e classes, e outra por parágrafos para o resto. O limite de 2000 caracteres é configurável com --max_chunk_size."*

Q4 da sheet: chunks demasiado pequenos perdem contexto (a função fica separada do seu docstring) e têm poucos termos para o BM25 pontuar; chunks demasiado grandes diluem o sinal, trazem ruído e podem não caber no contexto do LLM.

3.4 Demo de pesquisa (Q5)

`uv run python -m src search "How to configure OpenAI server?" --k 10`

#	Função (ficheiro)	O que faz
1	`RAGSystem.search()`	Ponto de entrada do comando.
2	`_validate_k()`	Rejeita k menor ou igual a 0 com mensagem e saída limpa.
3	verificação da query	Se estiver vazia ou só com espaços, escreve erro e sai. Não há traceback.
4	`_load_index()` -> `BM25Index.load()`	Confirma que bm25.pkl existe; carrega o modelo, os metadados e os textos e volta a juntar cada chunk ao seu texto pela posição.
5	`BM25Index.search()`	tokenize(query); bm25.get_scores; ordena por pontuação; fica com os k melhores; descarta pontuação 0 e intervalos repetidos.
6	impressão	Mostra para cada resultado: ficheiro, intervalo de caracteres e score.

> ***O que dizer:*** *"O BM25 devolve referências: ficheiro e intervalo de caracteres. O texto só é relido mais tarde pelo gerador."*

3.5 Demo de resposta (Q6)

`uv run python -m src answer "How to configure OpenAI server?" --k 10`

#	Função (ficheiro)	O que faz
1	`RAGSystem.answer()`	Valida k e query, carrega o índice e pesquisa k fontes (igual ao search).
2	`AnswerGenerator.__init__()` (generator.py)	Modelo por defeito **Qwen/Qwen3-0.6B** (é o que a Q6 pede). max_context_length=2000, max_sources=3, max_new_tokens=256.
3	`AnswerGenerator.load()`	Escolhe MPS, CUDA ou CPU; carrega tokenizer e modelo; põe em modo de avaliação sem gradientes.
4	`AnswerGenerator.generate()`	Chama _build_prompt, aplica o chat template (enable_thinking=False), gera sem amostragem, corta o prompt e descodifica só a resposta.
5	`_build_prompt()` e `_resolve_context_path()`	Para as **3 primeiras** fontes: encontra o ficheiro, lê content[start:end], respeita o limite de 2000 caracteres no total e monta [caminho] + texto. Acrescenta a instrução de usar só o contexto e a pergunta.

> ***O que dizer:*** *"Recupero 10 fontes, mas o gerador usa as 3 melhores e no máximo 2000 caracteres de contexto, para caber no modelo pequeno. Se o contexto não tiver a resposta, o prompt manda responder que não há informação suficiente."*

3.6 make exam-retrieval (Q2 e Q7a-Q7d)

O Makefile procura o exam_retrieval.sh e o binário moulinette e chama o script com `--student-path . --moulinette-path ... --module-name student`. Se não encontrar o script, imprime uma mensagem e salta. Resultados em `evaluations/retrieval/<data>/`. O script faz 4 testes:

Teste	O que corre	Critério	Funções tuas envolvidas
1. Indexação	python -m student index --max_chunk_size 2000	Até 300 s	Todas as de 3.3
2. Throughput	search_dataset sobre docs e code privados (unanswered), k=10, 200 perguntas no total	Até 90 s no total	search_dataset (ver abaixo)

Teste	O que corre	Critério	Funções tuas envolvidas
3. Docs Recall@5	moulinette evaluate_student_search_results sobre o JSON de docs	Pelo menos 0.80	Nenhuma: é o binário externo
4. Code Recall@5	O mesmo sobre o JSON de code	Pelo menos 0.50	Nenhuma: é o binário externo

Cadeia do search_dataset (teste 2):

#	Função (ficheiro)	O que faz
1	`RAGSystem.search_dataset()`	Recebe --dataset_path, --k e --save_directory.
2	`_validate_k()`, `_load_index()`	Valida k e carrega o índice uma só vez.
3	`_load_json_model(path, RagDataset)`	Lê o JSON e valida com o modelo RagDataset (aceita perguntas com ou sem resposta).
4	ciclo com tqdm	Para cada pergunta: BM25Index.search(question, k).
5	`MinimalSource` e `MinimalSearchResults`	Cada resultado vira MinimalSource; as fontes da pergunta ficam num MinimalSearchResults com question_id, question_str e retrieved_sources.
6	`StudentSearchResults` e `_write_json()`	Junta tudo com o k e grava o JSON com o mesmo nome do dataset, por isso o script encontra os ficheiros com *docs* e *code*.

Nos testes 3 e 4 o script procura a linha começada por `PASS:` e extrai `Recall@5 = ...` para as estrelas (docs: 80/85/90/95%; code: 50/55/60/65%). O teu comando `evaluate` (com _compute_recall_at_k, _source_found e _iou) **não é chamado** por este script; serve para ensaiares localmente. A tua regra é: mesmo ficheiro e IoU de intervalos de pelo menos 0.05. **A regra exata do moulinette não é visível para mim.** O teu README declara 0.840 (docs) e 0.590 (code) com o binário oficial.

> ***O que dizer:*** *"O recall mede, para cada pergunta, que fração das fontes certas aparece nos primeiros 5 resultados. Uma fonte conta como encontrada se for do mesmo ficheiro e o intervalo de caracteres se sobrepuser." (Confirma esta última frase com a tua avaliação local; é a regra do teu evaluate.)*

3.7 make exam-answer (Q8 e Q8b)

Corre o exam_answer.sh, que é semiautomático e **interativo**:

Passo do script	O que acontece	Funções tuas
1	Procura o diretório search_results mais recente em evaluations/retrieval (por isso corre primeiro o exam-retrieval; senão usa data/output/search_results). Chama moulinette list_valid_questions com k=10 e mostra as perguntas cujas fontes foram bem recuperadas.	Nenhuma (binário externo, sobre o JSON que search_dataset gravou)
2	Pede-te para escreveres 3 perguntas, copiadas da lista [VALID].	Nenhuma
3	Para cada pergunta corre python -m student answer "pergunta" --k 10 e guarda a saída em evaluations/answer/<data>/answer_N.log.	Cadeia de 3.5: RAGSystem.answer, AnswerGenerator.load/generate/_build_prompt
4	Tu julgas: passa se 2 de 3 respostas forem satisfatórias.	—

Para saltar a escrita interativa, corre o script diretamente com `--questions "pergunta 1,pergunta 2,pergunta 3"` (o target make não passa esta opção).

Critérios da Q8: a resposta deve ser autónoma, ancorada na fonte, fiel e relevante. Ver o ponto de atenção sobre citação de fontes na secção 5.

3.8 make exam-edge-cases (Q11)

Corre o exam_edge_cases.sh. Executa 4 comandos, ignora o código de saída e só falha se aparecer `Traceback (most recent call last)` no stderr. Logs em evaluations/edge_cases/<data>/.

#	Comando testado	O que o teu código faz	Resultado
1	search '' --k 10	RAGSystem.search deteta query vazia, escreve erro em stderr e sai com sys.exit(1).	Sem traceback
2	search 'asdfghjkl zxcvbnm qwertyuiop' --k 10	BM25Index.search dá pontuação 0 a tudo; essas são descartadas; devolve []; imprime "No results found."	Sem traceback
3	answer 'What is vLLM?' --k 0	_validate_k(0) sai antes de carregar índice ou modelo.	Sem traceback
4	search_dataset --dataset_path /nonexistent/dataset.json --k 10	_load_index corre primeiro; depois _load_json_model apanha FileNotFoundError e escreve "file not found". Se o índice não existir, sai com "Index not found".	Sem traceback

> ***O que dizer:*** *"Valido a entrada logo no início de cada comando e uso mensagens de erro com saída limpa em vez de deixar exceções subir. Por isso o script não deteta nenhum traceback."*

4. Exemplo de uma pergunta do início ao fim

Pergunta: **How to configure OpenAI server?**. Os ficheiros, offsets e scores são **ilustrativos**; os tokens da pergunta são o que a tua função tokenize devolve de facto.

```
1) tokenize("How to configure OpenAI server?")
   -> ['how','how','to','to','configure','configure','openai','openai','server','server']

2) BM25Index.search(..., k=10)  (valores ilustrativos)
   [{'file_path': 'data/raw/vllm-0.10.1/docs/.../openai_compatible_server.md',
     'first_character_index': 1200, 'last_character_index': 3150, 'score': 14.2}, ... ]

3) search_dataset grava (StudentSearchResults):
   {"k": 10, "search_results": [{"question_id": "...", "question_str": "...",
     "retrieved_sources": [{"file_path": "...", "first_character_index": 1200,
                            "last_character_index": 3150}, ...]}]}

4) generate(): _build_prompt le content[1200:3150] das 3 melhores fontes (max. 2000 chars),
   monta "Use only the provided context ... Context: [...] Question: ... Answer:"
   -> Qwen3-0.6B -> texto da resposta

5) answer_dataset grava (StudentSearchResultsAndAnswer): o mesmo JSON, com "answer" em cada item
```

Extra: make answer_dataset

Não faz parte dos 6 comandos nem dos exam scripts, mas é onde se gera o `StudentSearchResultsAndAnswer`. Lê os JSON de pesquisa de data/output/search_results, e para cada pergunta chama `AnswerGenerator.generate`, cria um `MinimalAnswer` e grava o lote com `_write_json`.

5. Pontos de atenção (encontrados no código)

Coisas reais que um avaliador pode notar. Verifica-as antes da defesa.

- **make install apaga o uv.lock** e gera outro. A sheet exige uv.lock presente; confirma que o ficheiro que entregas é o que queres.
- **Citação de fontes (Q8).** A sheet diz que uma resposta satisfatória cita as fontes. No teu código o prompt inclui [caminho] antes de cada trecho, mas não instrui o modelo a citar, e o comando answer imprime só "Answer: ...", sem listar as fontes. Prepara-te para explicar isto ou para mostrar as fontes com o comando search.
- **tokenize e camelCase.** O texto passa para minúsculas antes da regex que separa camelCase, por isso essa separação nunca dispara (testei: OpenAICompatibleServer fica como um único token). O snake_case funciona. O teu README fala em dividir camelCase, e isso não acontece. Além disso, palavras simples aparecem duplicadas na lista (uma vez como parte e outra como token completo).
- **Só 3 fontes e 2000 caracteres** chegam ao LLM, apesar de k=10. É uma decisão consciente; sabe justificá-la.
- **exam-answer depende do exam-retrieval**: usa o search_results mais recente. Sem ele, cai para data/output/search_results.
- **exam-retrieval re-indexa** com max_chunk_size 2000 e substitui o índice que tinhas.
- **Recall do moulinette vs. o teu evaluate:** o binário é que conta na sheet. Os teus números locais são só uma estimativa.

6. Respostas-base para a Q9 (adapta às tuas palavras)

Pergunta da sheet	Resposta apoiada no teu projeto
1. O que é RAG e porque é útil?	O modelo gera texto a partir de documentos recuperados, em vez de depender só do que memorizou. Reduz alucinações e permite responder sobre dados que ele não viu, como o código do vLLM.
2. Pipeline completo	Documentos (vLLM) -> chunker (AST para .py, parágrafos para o resto, máx. 2000) -> tokenize -> BM25 guardado em disco. Pergunta -> tokenize -> scores BM25 -> top-k com ficheiro e offsets -> gerador relê content[start:end] das 3 melhores -> prompt -> Qwen3-0.6B -> resposta.
3. TF-IDF vs BM25	Ambos pesam termos raros. O BM25 acrescenta saturação da frequência do termo (repetir uma palavra muitas vezes tem retorno decrescente) e normalização pelo comprimento do documento. Usas BM25Okapi do rank_bm25.
4. Escolhas e trade-offs	Chunking por AST preserva funções inteiras mas gera chunks de tamanhos desiguais. BM25 é rápido e sem GPU, mas não apanha sinónimos. Contexto de 2000 caracteres e 3 fontes cabem no modelo pequeno, mas podem perder informação.
5. O que melhorarias	Pesquisa híbrida BM25 + embeddings, expansão de query, corrigir a separação de camelCase, instruir o modelo a citar fontes, cache de queries.